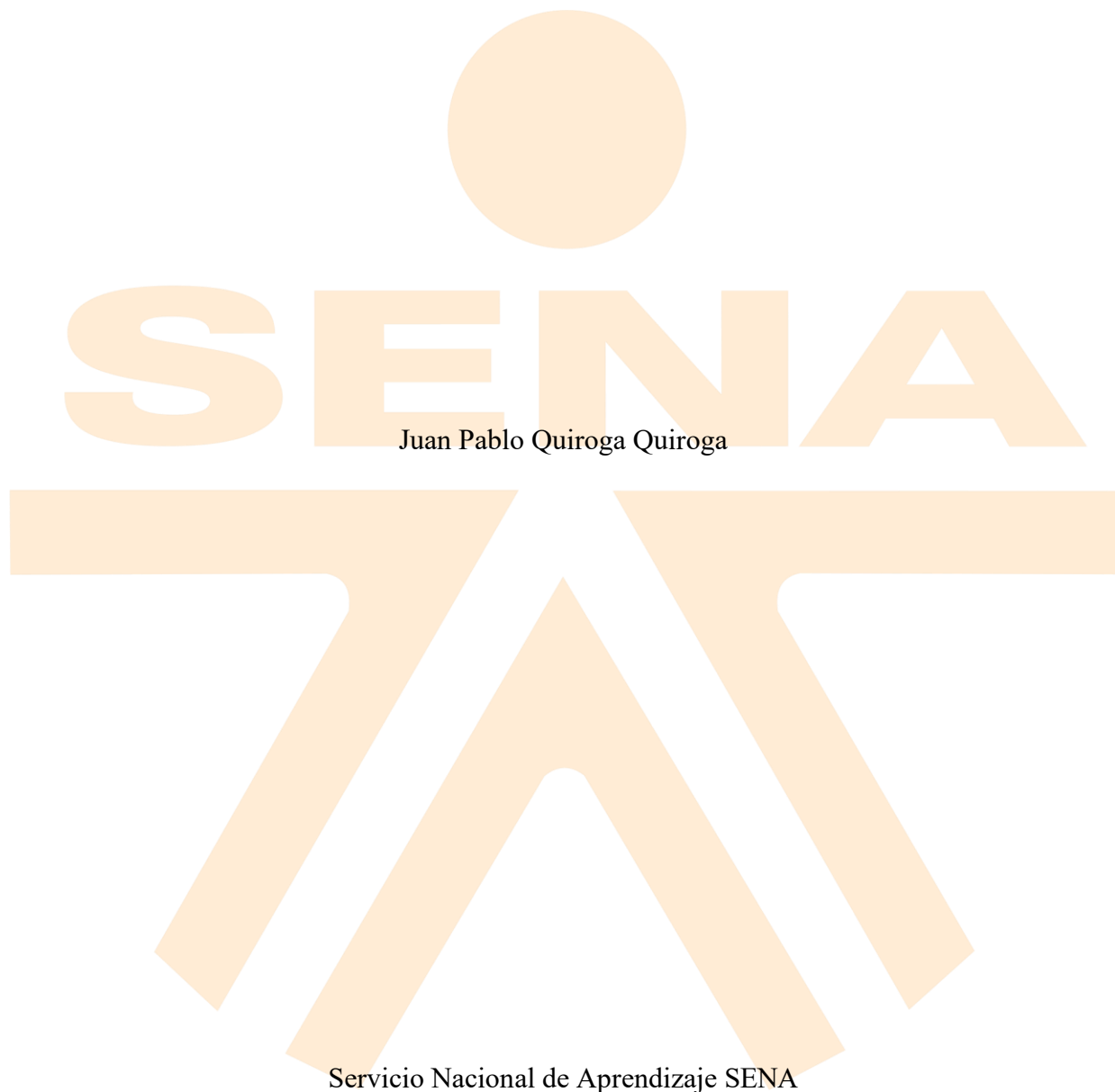


Codificar los Módulos del Software Stand-alone, Web y Móvil y Codificar el Front-end Utilizando Framework



Juan Pablo Quiroga Quiroga

Servicio Nacional de Aprendizaje SENA

Tecnólogo en Análisis y Desarrollo de Software Ficha – 3118562

Instructor: Henry Alfonso Garzón Sanchez

Contenido

Introducción	3
Objetivo	4
Sección 1 Implementación de Java con Hibernate	5
Paso 1 – Instalación de Dependencias de Hibernate	5
Paso 2 – Crear el Archivo (hibernate.cfg.xml)	6
Paso 3 – Anotar la Clase (Estudiante) con JPA (Java Persistence API)	8
Registrar la Entidad en (hibernate.cfg.xml)	10
Paso 4 – Crear (HibernateUtil.java)	11
Paso 5 – Reescribir el (Estudiantes.DAO.java) con Hibernate	13
Sección 2 Componentes Front-end	19
Diferencia Entre React y JSX	19
¿Qué son Clases en React?	21
Principales Eventos en React	22
Eventos del ratón	22
Eventos de Formularios y Teclado	23

Introducción

El presente documento presenta el desarrollo de la evidencia para la implementación del framework Hibernate en una aplicación web Java, tomando como base al ejercicio realizado previamente, en el cual se construyó un CRUD (Create, Read, Update, Delete) de estudiantes utilizando Servlets, JSP (JavaServer Pages) y JDBC (Java Database Connectivity) como capa de acceso a datos. El objetivo de esta nueva evidencia consiste en implementar esta misma aplicación para que su comunicación con la base de datos se realice mediante Hibernate, un framework ORM (Object-Relational Mapping) que automatiza el mapeo entre las clases Java y las tablas relacionales de SQL Server, reemplazando las consultas SQL (Structured Query Language) y la gestión manual de conexiones por una forma más simple.

Se creó un proyecto independiente llamado crud-web-hibernate, en el cual se reutilizaron las capas que no requerían modificaciones — Servlets, JSPs, hojas de estilo CSS (Cascading Style Sheets) y la estructura general del proyecto anterior, se reescribió únicamente la capa de acceso a datos, junto con la configuración necesaria para integrar Hibernate. La base de datos ejemplo_colegio en SQL Server se mantuvo igual, asegurando que el cambio fuese exclusivamente en de la lógica de acceso.

Se desarrollará el paso a paso para esta implementación detallando los cambios realizados con respecto a la anterior evidencia y mostrando el resultado tras el desarrollo.

Objetivo

Codificar los módulos del sistema (CRUD) haciendo uso del framework de Java – Hibernate plasmándolo en un paso a paso del proceso de implementación.



Sección 1 Implementación de Java con Hibernate

Tomando como base el proyecto que ya desarrollamos para Java web con JSP y Servlets, ahora lo que vamos a realizar es este mismo ejercicio haciendo uso de Hibernate ORM el cual es un framework de Java que nos permite realizar la integración de y mapeo de los objetos que tenemos en nuestra BD con Java.

Es una forma de trabajar con objetos Java en vez de escribir tanto código SQL a mano. La idea central de este framework es el ORM (Object Relational Model), el cual traduce a los objetos Java como tablas en SQL, Hibernate se encarga de hacer esta conexión automáticamente.

Paso 1 – Instalación de Dependencias de Hibernate

Para poder ejecutar nuestro proyecto dentro del entorno de Hibernate debemos hacer uso de 3 librerías:

- **Hibernate core.** Trae toda la lógica del ORM para el mapeo de las clases a tablas, la gestión de sesiones y transacciones.
- **Mssql-jdbc.** Este ya lo teníamos instalado que es el driver necesario para conectarnos en este caso con SQL Server.
- **Slf4j-simple.** Hibernate internamente genera muchos logs para que podamos ver que sucede por debajo de la aplicación, esta librería permite que esos logs aparezcan en la consola y tengamos un control más detallado del entorno de ejecución.

Dentro de nuestro archivo pom.xml vamos a añadir estas librerías adicionales a las que ya teníamos anteriormente.

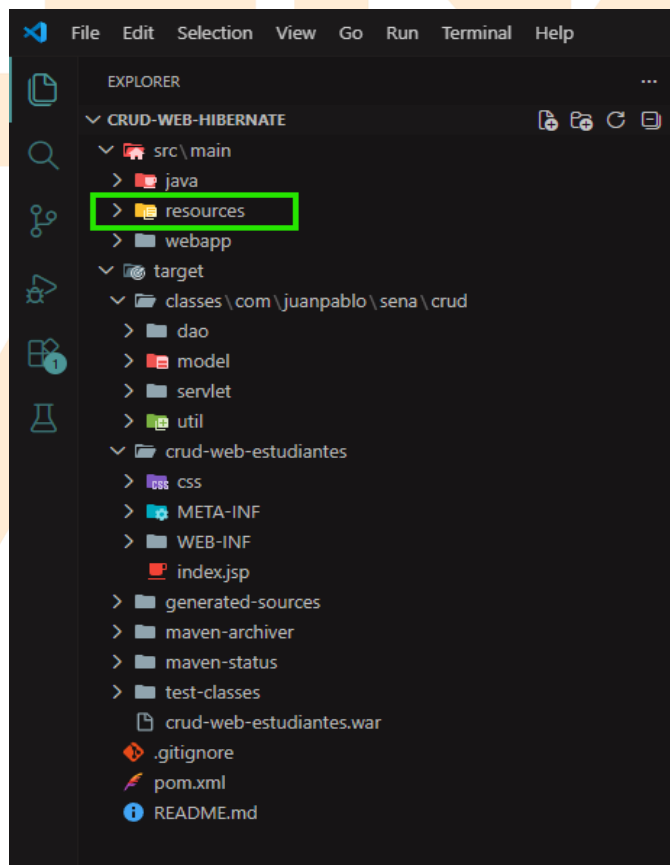
```
pom.xml
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
32  <dependencies>
49  </dependencies>
50  <!-- Hibernate -->
51  <dependency>
52      <groupId>org.hibernate.orm</groupId>
53      <artifactId>hibernate-core</artifactId>
54      <version>6.4.4.Final</version>
55  </dependency>
56
57  <!-- Logger simple para que Hibernate imprima logs limpios -->
58  <dependency>
59      <groupId>org.slf4j</groupId>
60      <artifactId>slf4j-simple</artifactId>
61      <version>2.0.13</version>
62  </dependency>
63
64  <!-- Driver JDBC (Java Database Connectivity) para SQL Server -->
65  <dependency>
66      <groupId>com.microsoft.sqlserver</groupId>
67      <artifactId>mssql-jdbc</artifactId>
68      <version>12.8.1.jre11</version>
69  </dependency>
70
71  </dependencies>
72
73  <build>
74      <finalName>crud-web-estudiantes</finalName>
```

Paso 2 – Crear el Archivo (hibernate.cfg.xml)

Este es un archivo de configuración en el que le decimos a Hibernate principalmente:

- A qué BD se debe conectar (URL, usuario y contraseña).
- Qué driver JDBC usar (SQL Server).
- El dialecto SQL que debe utilizar, ya que aunque SQL es un estándar, dentro de cada motor de BD hay pequeños cambios de sintaxis que difieren entre sí.
- Qué clases de Java debe mapear como entidades (nuestro modelo **Estudiante** en este caso).

Dentro de nuestra estructura de proyecto, vamos a crear una carpeta llamada **resources** dentro de **main** en donde estará almacenado este archivo.



Dentro de esta carpeta creamos el archivo junto con las configuraciones mencionadas:

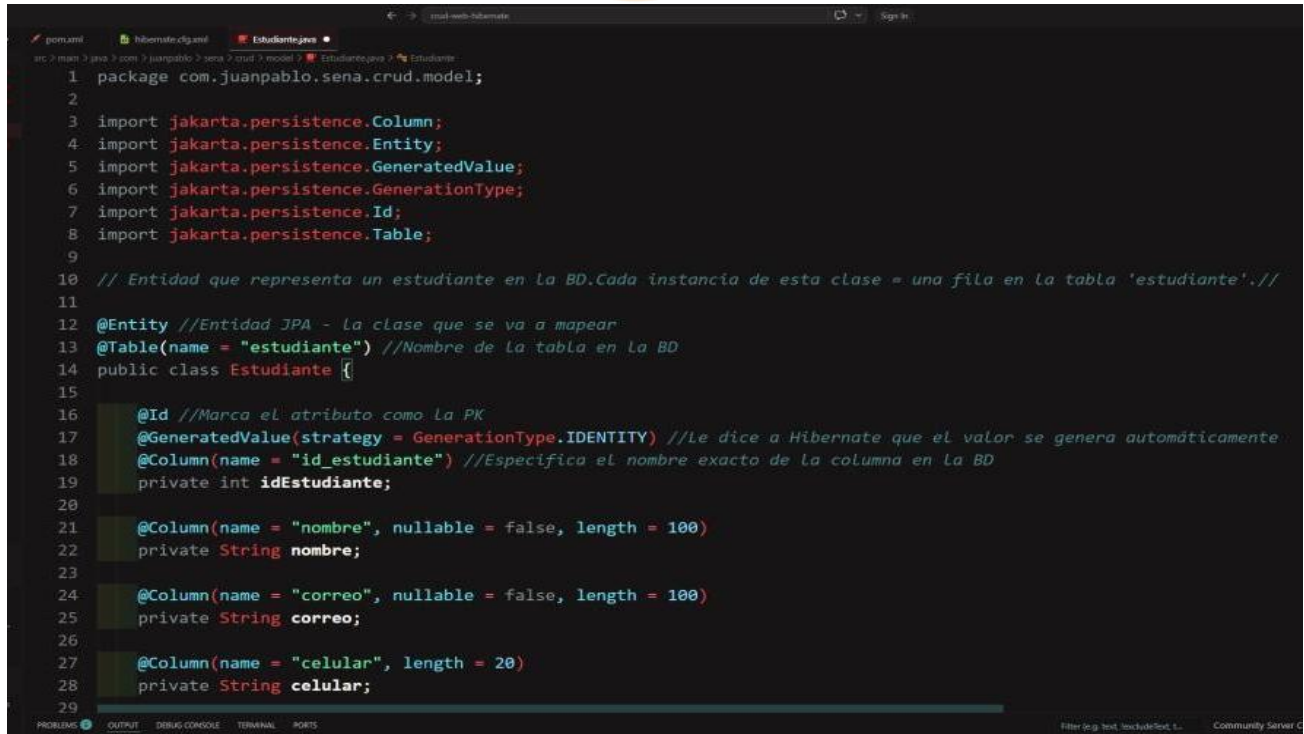
```
hibernate.cfg.xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE hibernate-configuration PUBLIC
3     "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4     "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
5
6 <hibernate-configuration>
7     <session-factory>
8
9         <!-- Conexión a La BD -->
10        <property name="hibernate.connection.driver_class">
11            com.microsoft.sqlserver.jdbc.SQLServerDriver
12        </property>
13
14        <property name="hibernate.connection.url">
15            jdbc:sqlserver://localhost:1433;databaseName=ejemplo_colegio;encrypt=true;trustServerCertificate=true
16        </property>
17
18        <property name="hibernate.connection.username">"sa"</property>
19        <property name="hibernate.connection.password">"sa5279#"</property>
20
21        <!-- Dialecto SQL para SQL Server -->
22        <property name="hibernate.dialect">
23            org.hibernate.dialect.SQLServerDialect
24        </property>
25
26        <!-- Comportamiento -->
27
28        <!-- Imprime en consola el SQL que Hibernate genera -->
29        <property name="hibernate.show_sql">true</property>
```

```
hibernate.cfg.xml
6 <hibernate-configuration>
7     <session-factory>
25
26     <!-- Comportamiento -->
27
28     <!-- Imprime en consola el SQL que Hibernate genera -->
29     <property name="hibernate.show_sql">true</property>
30
31     <!-- Formatea ese SQL para que sea legible -->
32     <property name="hibernate.format_sql">true</property>
33
34     <!-- Controla si Hibernate modifica el esquema de La BD, en este caso las tablas ya estan creadas -
35     dejamos La opción de validate para ver que coincidan con la clase en Java-->
36     <property name="hibernate.hbm2ddl.auto">validate</property>
37
38     </session-factory>
39 </hibernate-configuration>
```

Paso 3 – Anotar la Clase (Estudiante) con JPA (Java Persistence API)

Una anotación en Java es una etiqueta que empieza con **@** y se coloca encima de una clase, atributo o método para darle información adicional. Son metadatos que los frameworks usan para tomar decisiones en este caso Hibernate.

Para este paso debemos modificar la clase **Estudiante.java** de nuestro ejercicio anterior de la siguiente forma.

A screenshot of an IDE window showing the code for Estudiante.java. The code is in Spanish and includes JPA annotations for a database entity. The annotations include @Entity, @Table, @Id, @GeneratedValue, and @Column. The class has three private attributes: idEstudiante (int), nombre (String), correo (String), and celular (String).

```
1 package com.juanpablo.sena.crud.model;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.GeneratedValue;
6 import jakarta.persistence.GenerationType;
7 import jakarta.persistence.Id;
8 import jakarta.persistence.Table;
9
10 // Entidad que representa un estudiante en la BD. Cada instancia de esta clase = una fila en la tabla 'estudiante'.//
11
12 @Entity //Entidad JPA - La clase que se va a mapear
13 @Table(name = "estudiante") //Nombre de la tabla en la BD
14 public class Estudiante {
15
16     @Id //Marca el atributo como la PK
17     @GeneratedValue(strategy = GenerationType.IDENTITY) //Le dice a Hibernate que el valor se genera automáticamente
18     @Column(name = "id_estudiante") //Especifica el nombre exacto de la columna en la BD
19     private int idEstudiante;
20
21     @Column(name = "nombre", nullable = false, length = 100)
22     private String nombre;
23
24     @Column(name = "correo", nullable = false, length = 100)
25     private String correo;
26
27     @Column(name = "celular", length = 20)
28     private String celular;
29 }
```

```
14 public class Estudiante {
27     @Column(name = "celular", length = 20)
28     private String celular;
29
30     @Column(name = "direccion", length = 200)
31     private String direccion;
32
33     // Constructor vacío: OBLIGATORIO para JPA
34
35     public Estudiante() {
36     }
37
38     // Constructor con parámetros (opcional, útil para crear instancias rápido)
39     public Estudiante(String nombre, String correo, String celular, String direccion) {
40         this.nombre = nombre;
41         this.correo = correo;
42         this.celular = celular;
43         this.direccion = direccion;
44     }
45
46     // =====
47     // Getters y Setters
48     // =====
49     public int getIdEstudiante() {
50         return idEstudiante;
51     }
52
53     public void setIdEstudiante(int idEstudiante) {
54         this.idEstudiante = idEstudiante;
55     }
56 }
```

Los getters y setters se mantienen igual.

Para este paso es importante que declaremos un constructor de la clase vacío como ya lo teníamos, eso es necesario ya que cuando Hibernate trae cada fila de la BD necesita crear una instancia de esta clase vacía, por lo que va a tomar todas las filas en SQL Server y llenar esta instancia. De lo contrario va a generar un error.

```
14 public class Estudiante {
33     // Constructor vacío: OBLIGATORIO para JPA
34
35     public Estudiante() {
36     }
37
38     // Constructor con parámetros (opcional, útil para crear instancias rápido)
39     public Estudiante(String nombre, String correo, String celular, String direccion) {
40         this.nombre = nombre;
41         this.correo = correo;
42         this.celular = celular;
43         this.direccion = direccion;
44     }
45
46     // =====
47     // Getters y Setters
48     // =====
49     public int getIdEstudiante() {
50         return idEstudiante;
51     }
52
53     public void setIdEstudiante(int idEstudiante) {
54         this.idEstudiante = idEstudiante;
55     }
56
57     public String getNombre() {
58         return nombre;
59     }
60 }
```

Registrar la Entidad en (hibernate.cfg.xml)

Para que Hibernate rastree esta entidad que acabamos de definir tenemos que listarla explícitamente en el archivo de configuración.

Dentro del archivo indicamos la ruta exacta del paquete donde se encuentre alojada la clase **Estudiante.java**.

```
hibernate.cfg.xml
6 <hibernate-configuration>
7   <session-factory>
25
26   <!-- Comportamiento -->
27
28   <!-- Imprime en consola el SQL que Hibernate genera -->
29   <property name="hibernate.show_sql">true</property>
30
31   <!-- Formatea ese SQL para que sea legible -->
32   <property name="hibernate.format_sql">true</property>
33
34   <!-- Controla si Hibernate modifica el esquema de La BD, en este caso las tablas ya estan creadas -
35   dejamos La opción de validate para ver que coincidan con La clase en Java-->
36   <property name="hibernate.hbm2ddl.auto">validate</property>
37
38   <!--Clase mapeado como entidad-->
39   <mapping class="com.juanpablo.sena.crud.model.Estudiante"/>
40
41 </session-factory>
42 </hibernate-configuration>
```

Paso 4 – Crear (HibernateUtil.java)

Se creó la clase HibernateUtil.java. Esta clase centraliza la creación del SessionFactory de Hibernate aplicando el patrón Singleton, que garantiza que exista una sola instancia de esta clase durante toda la ejecución de la aplicación y que la misma se pueda reutilizar por todos los DAO.

```
1 package com.juanpablo.sena.crud.util;
2
3 import org.hibernate.SessionFactory;
4 import org.hibernate.boot.registry.StandardServiceRegistry;
5 import org.hibernate.boot.registry.StandardServiceRegistryBuilder;
6 import org.hibernate.cfg.Configuration;
7
8 /* Clase que gestiona el ciclo de vida del SessionFactory de Hibernate.
9  * Implementa el patrón Singleton: una sola SessionFactory por toda la aplicación,
10  * creada al cargar la clase y reutilizada por todos los DAOs.
11  */
12 public class HibernateUtil {
13
14     // Instancia única de SessionFactory (el "Singleton")
15     private static final SessionFactory sessionFactory = buildSessionFactory();
16
17     /*
18     * Método privado que construye el SessionFactory.
19     * Se ejecuta UNA SÓLA VEZ, cuando Java carga la clase por primera vez.
20     */
21     private static SessionFactory buildSessionFactory() {
22         try {
23             // 1. Configuration lee el archivo hibernate.cfg.xml
24             Configuration configuration = new Configuration().configure();
25
26             // 2. Construye el "service registry" (registro de servicios internos) aplicando las propiedades de
27             // configuración
28             StandardServiceRegistry serviceRegistry = new StandardServiceRegistryBuilder().applySettings(configuration.
29                 getProperties()).build();
30
31             // 3. Con la config + el registry, construye la SessionFactory
32             return configuration.buildSessionFactory(serviceRegistry);
33
34         } catch (Throwable ex) {
35             // Si algo falla (BD caída, archivo mal escrito, etc.), se ve en la consola con la clase que importamos.
36             System.err.println("Error al inicializar SessionFactory: " + ex);
37             throw new ExceptionInInitializerError(ex);
38         }
39     }
40
41     /*
42     * Único método público: devuelve la SessionFactory para que los DAOs la usen.
43     */
44     public static SessionFactory getSessionFactory() {
45         return sessionFactory;
46     }
47
48     // Método para cerrar la SessionFactory cuando la app se apaga.
49     public static void shutdown() {
50         if (sessionFactory != null && !sessionFactory.isClosed()) {
51             sessionFactory.close();
52             System.out.println("SessionFactory cerrada correctamente.");
53         }
54     }
55 }
```

```
12 public class HibernateUtil {
13     private static SessionFactory buildSessionFactory() {
14         try {
15             // 1. Configuration lee el archivo hibernate.cfg.xml
16             Configuration configuration = new Configuration().configure();
17
18             // 2. Construye el "service registry" (registro de servicios internos) aplicando las propiedades de
19             // configuración
20             StandardServiceRegistry serviceRegistry = new StandardServiceRegistryBuilder().applySettings(configuration.
21                 getProperties()).build();
22
23             // 3. Con la config + el registry, construye la SessionFactory
24             return configuration.buildSessionFactory(serviceRegistry);
25
26         } catch (Throwable ex) {
27             // Si algo falla (BD caída, archivo mal escrito, etc.), se ve en la consola con la clase que importamos.
28             System.err.println("Error al inicializar SessionFactory: " + ex);
29             throw new ExceptionInInitializerError(ex);
30         }
31     }
32
33     /*
34     * Único método público: devuelve la SessionFactory para que los DAOs la usen.
35     */
36     public static SessionFactory getSessionFactory() {
37         return sessionFactory;
38     }
39
40     // Método para cerrar la SessionFactory cuando la app se apaga.
41     public static void shutdown() {
42         if (sessionFactory != null && !sessionFactory.isClosed()) {
43             sessionFactory.close();
44             System.out.println("SessionFactory cerrada correctamente.");
45         }
46     }
47 }
```

```
12 public class HibernateUtil {
13
14     /*
15     * Único método público: devuelve La SessionFactory para que los DAOs la usen.
16     */
17     public static SessionFactory getSessionFactory() {
18         return sessionFactory;
19     }
20
21     // Método para cerrar la SessionFactory cuando la app se apaga.
22     public static void shutdown() {
23         if (sessionFactory != null && !sessionFactory.isClosed()) {
24             sessionFactory.close();
25             System.out.println(x: "SessionFactory cerrada correctamente.");
26         }
27     }
28 }
29 }
```

El SessionFactory se construye una única vez al cargar la clase, leyendo automáticamente la configuración definida en hibernate.cfg.xml. A partir de ese momento, cualquier parte del proyecto puede obtener sesiones de Hibernate llamando a `HibernateUtil.getSessionFactory()`.

Paso 5 – Reescribir el (Estudiantes.DAO.java) con Hibernate

En este paso se reemplazó por completo la implementación anterior de la clase `EstudianteDAO.java`. La versión vieja utilizaba JDBC, lo que implicaba abrir manualmente la conexión a través de la clase `Conexion`, escribir las consultas SQL como cadenas de texto, gestionar `PreparedStatement` para insertar parámetros, recorrer los `ResultSet` fila por fila para construir los objetos `Estudiante`, y cerrar manualmente todos los recursos.

En este ejercicio se implementa el framework Hibernate con la clase HibernateUtil creada en el paso anterior, para obtener sesiones de una forma más centralizada haciendo uso de una misma clase. Cada método del CRUD sigue ahora una estructura en la que se abre una Session mediante try-with-resources (lo que garantiza su cierre automático), se inicia una Transaction (transacción) cuando la operación modifica datos, se ejecuta la acción correspondiente y se confirma con commit. En caso de error, se ejecuta un rollback que revierte cualquier cambio parcial, lo que nos permite mantener segura la información de la base de datos evitando cambios a medias.

El CRUD para este caso quedo así:

Insertar: se utiliza `session.persist(estudiante)`, que genera el INSERT automáticamente a partir de las anotaciones JPA de la entidad. El ID que genera automáticamente SQL Server se asigna de vuelta al objeto Java después de la inserción.

Listar todos: se utiliza HQL (Hibernate Query Language) con la consulta FROM Estudiante. A diferencia del SQL normal, HQL funciona sobre el nombre de la clase Java, no sobre el de la tabla. Hibernate se encarga de traducir la consulta al SQL específico de SQL Server y de instanciar automáticamente los objetos Estudiante con los datos recuperados.

Buscar por ID: se utiliza `session.get(Estudiante.class, id)`, que recupera directamente la fila correspondiente a la clave primaria. Devuelve null si no existe el registro, esto nos ayuda a manejar los casos en que el ID buscado no exista.

Actualizar: se utiliza `session.merge(estudiante)`. Este método funciona, aunque el objeto Estudiante venga desde afuera del contexto de Hibernate (por ejemplo, actualizado a partir de los parámetros del formulario). Hibernate detecta automáticamente los campos que cambiaron y genera el UPDATE.

Eliminar: primero se localiza el registro con `session.get()` y luego se elimina con `session.remove()`. Si el registro no existe, no se ejecuta ninguna acción.

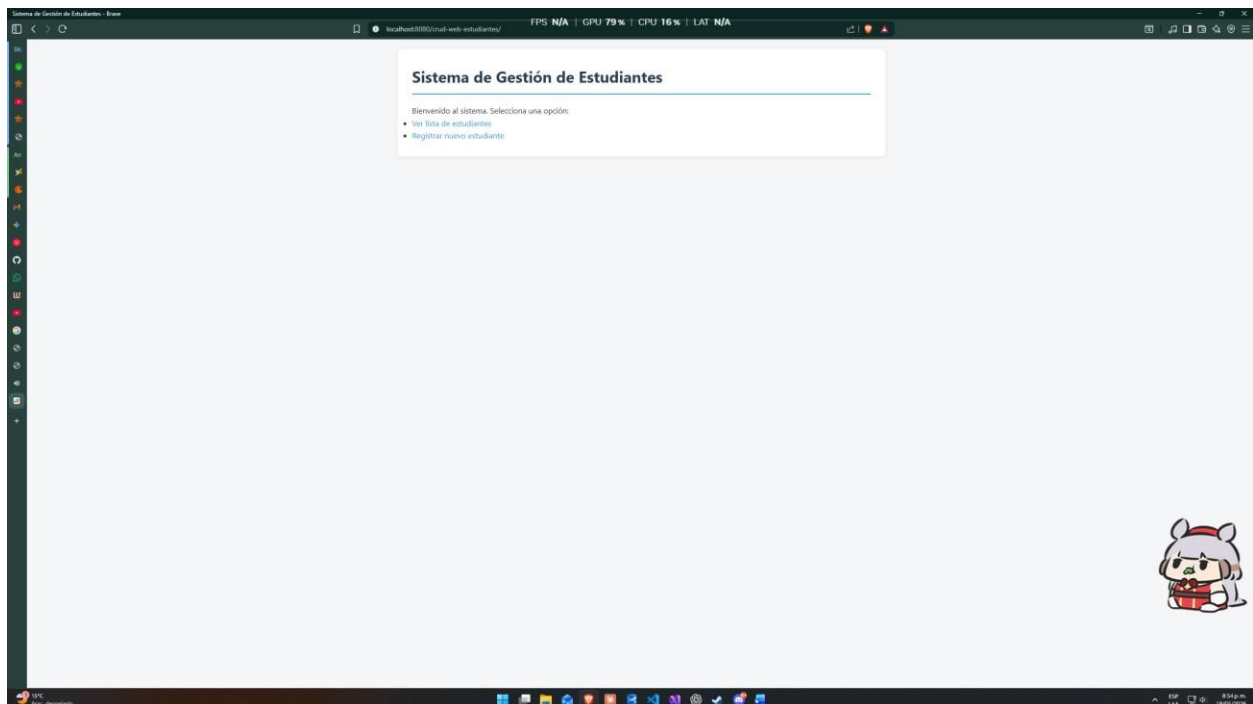
```
src > main > java > com > juanpablo.sena > crud > dao > EstudianteDAO.java > EstudianteDAO

4 import com.juanpablo.sena.crud.util.HibernateUtil;
5 import org.hibernate.Session;
6 import org.hibernate.Transaction;
7 import org.hibernate.query.Query;
8
9 import java.util.List;
10
11 /*
12  * DAO de Estudiante usando Hibernate.
13  * Reemplaza la versión antigua basada en JDBC + Conexion.java.
14  */
15 public class EstudianteDAO {
16
17     // CREATE: insertar un nuevo estudiante
18     public void insertar(Estudiante estudiante) {
19         try (Session session = HibernateUtil.getSessionFactory().openSession()) {
20             Transaction tx = session.beginTransaction();
21             try {
22                 session.persist(estudiante);
23                 tx.commit();
24             } catch (Exception e) {
25                 tx.rollback();
26                 throw e;
27             }
28         }
29     }
30 }
```

```
15 public class EstudianteDAO {
31     // READ: Listar todos los estudiantes
32     public List<Estudiante> listarTodos() {
33         try (Session session = HibernateUtil.getSessionFactory().openSession()) {
34             Query<Estudiante> query = session.createQuery(
35                 queryString: "FROM Estudiante", resultClass: Estudiante.class
36             );
37             return query.list();
38         }
39     }
40
41     // READ con ID: buscar un estudiante por su ID
42     public Estudiante buscarPorId(int id) {
43         try (Session session = HibernateUtil.getSessionFactory().openSession()) {
44             return session.get(entityType: Estudiante.class, id);
45         }
46     }
47
48     // UPDATE: actualizar un estudiante
49     public void actualizar(Estudiante estudiante) {
50         try (Session session = HibernateUtil.getSessionFactory().openSession()) {
51             Transaction tx = session.beginTransaction();
52             try {
53                 session.merge(estudiante);
54                 tx.commit();
55             } catch (Exception e) {
56                 tx.rollback();
57                 throw e;
58             }
59         }
60     }
61 }
```

```
15 public class EstudianteDAO {
49     public void actualizar(Estudiante estudiante) {
57         throw e;
58     }
59 }
60 }
61
62 // DELETE: eliminar un estudiante por su ID
63 public void eliminar(int id) {
64     try (Session session = HibernateUtil.getSessionFactory().openSession()) {
65         Transaction tx = session.beginTransaction();
66         try {
67             Estudiante estudiante = session.get(entityType: Estudiante.class, id);
68             if (estudiante != null) {
69                 session.remove(estudiante);
70             }
71             tx.commit();
72         } catch (Exception e) {
73             tx.rollback();
74             throw e;
75         }
76     }
77 }
78 }
```

Con esto terminaríamos el proceso, solo nos queda ver el resultado en nuestro localhost:



Lista de Estudiantes

Volver al inicio

Registrar nuevo estudiante

Total: 21 estudiantes

ID	Nombre	Correo	Celular	Dirección	Acciones
1	Juan Pablo Ramirez	juan.ramirez@gmail.com	3104567890	Calle 45 #12-34, Bogotá	Editar Eliminar
2	Maria Fernanda López	mariaf.lopez@hotmail.com	3209676543	Carera 15 #78-21, Medellín	Editar Eliminar
3	Carlos Andrés Gómez	cgomez@outlook.com	3156789012	Avenida 68 #34-56, Bogotá	Editar Eliminar
4	Laura Catalina Pérez	laura.perez@gmail.com	3018765432	Calle 80 #25-10, Cali	Editar Eliminar
6	Diana Marcela Ruiz	diana.ruiz@gmail.com	3145678901	Calle 100 #15-67, Bogotá	Editar Eliminar
8	Valentina Morales	vmorales@gmail.com	3187654321	Calle 26 #89-12, Medellín	Editar Eliminar
9	Daniel Esteban Rojas	danielrojas@outlook.com	3076543210	Avenida Caracas #56-78, Bogotá	Editar Eliminar
10	Camila Andrea Vargas	camila.vargas@gmail.com	3134567890	Carera 30 #67-45, Cali	Editar Eliminar
11	Mateo Alejandro Díaz	mateodiaz@yahoo.com	3198765432	Calle 53 #20-15, Bogotá	Editar Eliminar
12	Isabella Restrepo	isabella.r@gmail.com	3167890123	Carera 43 #10-98, Medellín	Editar Eliminar
13	Santiago Herrera	sherrera@hotmail.com	3045678901	Calle 72 #11-23, Bogotá	Editar Eliminar
14	Sofía Alejandra Mejía	sofia.mejia@gmail.com	3123456789	Avenida 19 #100-45, Bogotá	Editar Eliminar
15	Nicolás Sánchez	nsanchez@outlook.com	3056789012	Carera 9 #78-34, Cali	Editar Eliminar
16	Gabriela Ortiz	gabrielortiz@gmail.com	3189012345	Calle 116 #100-67, Bogotá	Editar Eliminar



Registrar nuevo estudiante

Volver al inicio

Ver Estado

Nombre:

Daniel Quintan

Correo:

dquintero@gmail.com

Celular:

3008763456

Dirección:

Diagonal 34 #4 - 88

Guardar



Lista de Estudiantes - Browser

localhost:3030/crud-web-estudiantes/estudiantes/lista

FPS N/A | GPU 2% | CPU 6% | LAT N/A

Spanish English

8	Valentina Morales	vmorales@gmail.com	3187654321	Calle 26 #89-12, Bogotá	Editar Eliminar
9	Daniel Esteban Rojas	danierojas@outlook.com	3076543210	Avenida Caracas #56-78, Bogotá	Editar Eliminar
10	Camila Andrea Vargas	camila.vargas@gmail.com	3134567890	Carera 30 #67-45, Cali	Editar Eliminar
11	Mateo Alejandro Diaz	mateodiaz@yahoo.com	3198765432	Calle 53 #20-15, Bogotá	Editar Eliminar
12	Isabella Restrepo	isabella.r@gmail.com	3167890123	Carera 43 #10-98, Medellín	Editar Eliminar
13	Santiago Herrera	sherrera@hotmail.com	3045678901	Calle 72 #11-23, Bogotá	Editar Eliminar
14	Sofia Alejandra Mejia	sofia.meja@gmail.com	3123456789	Avenida 19 #100-45, Bogotá	Editar Eliminar
15	Nicolás Sánchez	nsanchez@outlook.com	3056789012	Carera 9 #78-34, Cali	Editar Eliminar
16	Gabriela Ortiz	gabrielortiz@gmail.com	3189012345	Calle 116 #20-57, Bogotá	Editar Eliminar
17	Esteban Cárdenas	ecardenas@yahoo.com	3092345678	Carera 60 #45-12, Barranquilla	Editar Eliminar
18	Mariana Quintero	mquintero@gmail.com	3178901234	Calle 34 #56-78, Medellín	Editar Eliminar
19	Felipe Augusto Niño	felipe.nino@hotmail.com	3067890123	Avenida Boyack #80-23, Bogotá	Editar Eliminar
20	Paula Andrea Bermúdez	paula.bermudez@gmail.com	3145678902	Carera 11 #93-45, Bogotá	Editar Eliminar
21	Juan Quiruga Quiruga	jquirugaedit@gmail.com	3009999999	Nueva Calle 100	Editar Eliminar
22	Ginnette Alarcon	ginnettealarcon@gmail.com	3004560988	Calle 19 #4 - 62	Editar Eliminar
25	Liliana Gonzalez Sierra	llig@gmail.com	3458976789	Carera 34 #4 - 6	Editar Eliminar
26	Daniel Quintero	dquintero@gmail.com	3008763456	Diagonal 34 #4 - 89	Editar Eliminar



Editar estudiante - Browser

localhost:3030/crud-web-estudiantes/estudiantes/editar/26

FPS N/A | GPU 12% | CPU 4% | LAT N/A

Editar estudiante

[Volver al inicio](#) [Ver listado](#)

Nombre: Daniel Quintero

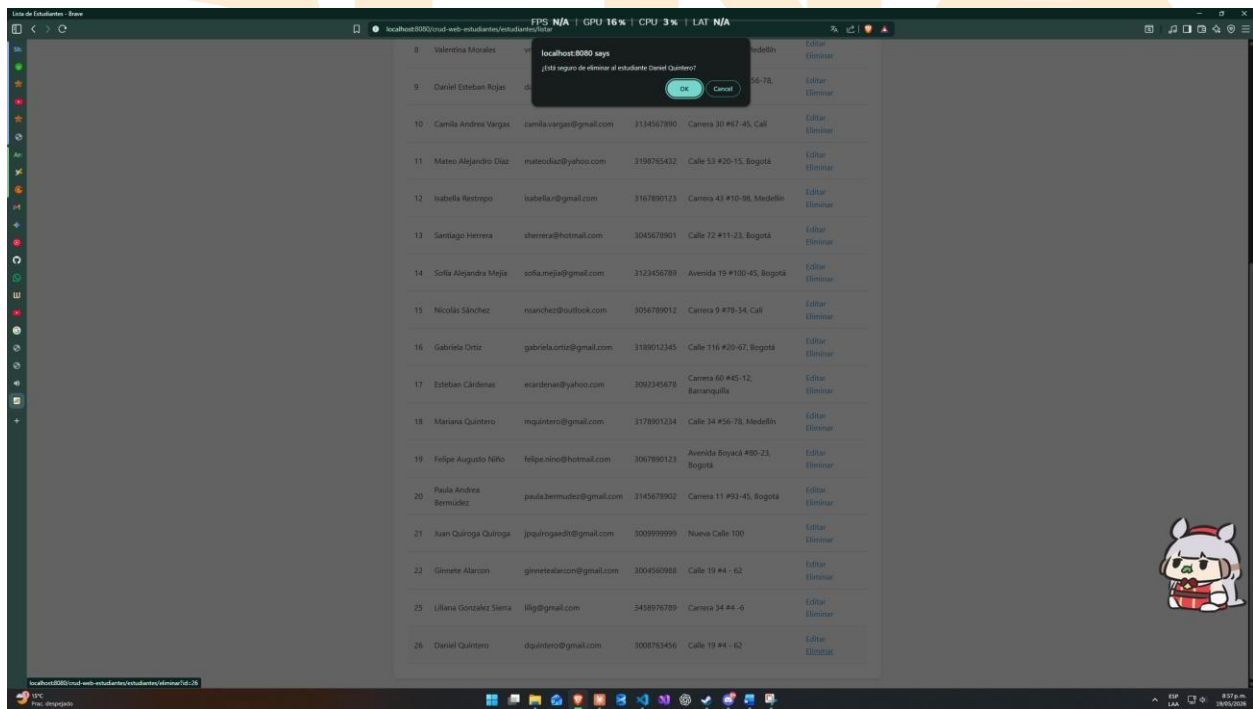
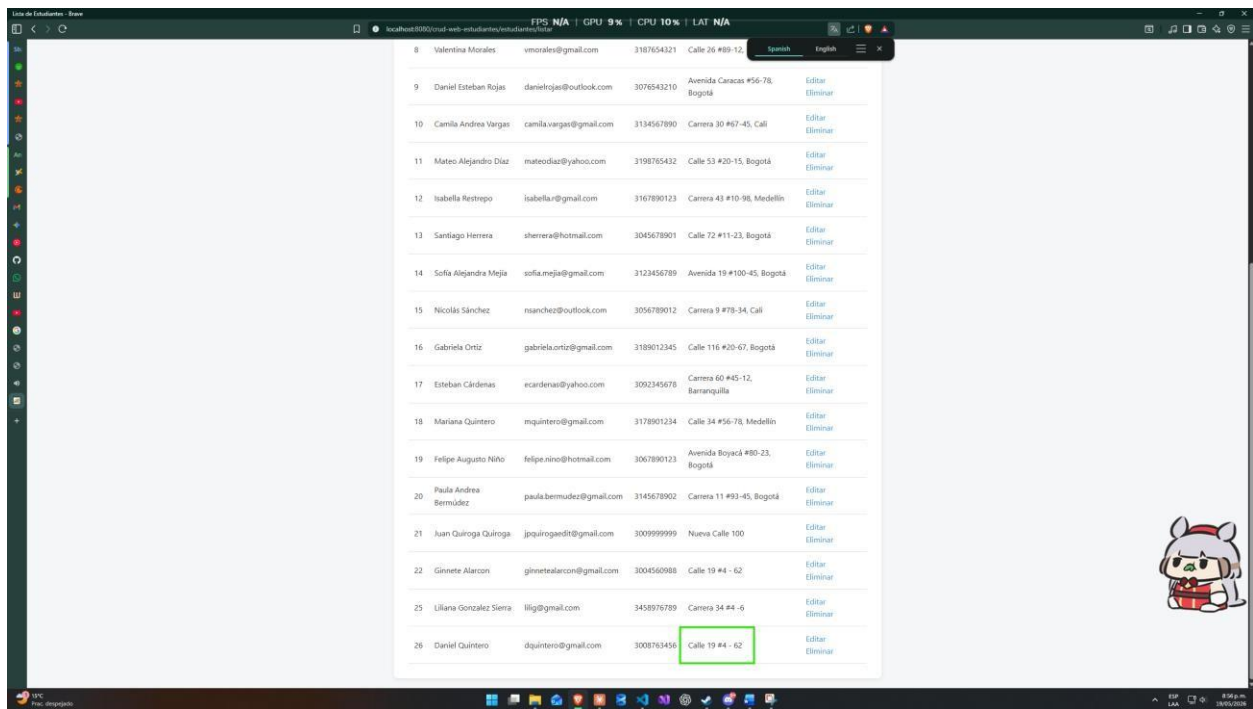
Correo: dquintero@gmail.com

Celular: 3008763456

Dirección: Calle 19 #4 - 62

[Actualizar](#)





Sección 2 Componentes Front-end

Diferencia Entre React y JSX

React es una biblioteca de JavaScript para el desarrollo de páginas web, específicamente SPA (Single Page Applications) que se basa en el uso de componentes.

React nos permite encapsular componentes tan pequeños de una página como lo pueden ser un botón, hasta algo tan grande como una página web completa. Con React podemos definir a estos componentes como elementos de uso reiterativo ya que los definimos de forma declarativa lo que nos permite reutilizarlos las veces que sean necesarias incluso dentro de otros componentes.

Un componente se define de la siguiente forma:

```
function MyButton() {  
  return (  
    <button>Soy un botón</button>  
  );  
}
```

Y este mismo componente lo podemos utilizar de forma anidada dentro de otro:

```
export default function MyApp() {  
  return (  
    <div>  
      <h1>Bienvenido a mi aplicación</h1>  
      <MyButton />  
    </div>  
  );  
}
```

Esta forma de declaración se conoce como **JSX (JavaScript XML)** que básicamente nos permite utilizar código HTML de la mano con JavaScript de una forma integrada. Es la sintaxis que se usa para trabajar con React.

La diferencia que tenemos a trabajar con HTML puro es que el uso de JSX es más estricto. Dentro de las reglas de uso de esta sintaxis tenemos:

- Podemos utilizar lógica JS dentro del HTML con el uso de llaves { } como mostrar el valor de una variable o para el uso de cálculos matemáticos.
- **class** pasa a ser **className** ya que class esta reservada para la definición de clases en JavaScript aunque su funcionamiento es fundamentalmente el mismo.
- Las funciones siempre deben retornar un único elemento “padre” o en caso de que necesitemos definir más de uno, debe estar dentro de un elemento raíz que los envuelva.
 - o Mal: `<h1>Título</h1><p>Párrafo</p>`
 - o Bien: `<div> <h1>Título</h1> <p>Párrafo</p> </div>`

Con esto claro podemos concluir que JSX es la sintaxis con la que comúnmente trabajamos con la biblioteca de React y es el lenguaje de marcado que utilizan la mayoría de proyectos por su practicidad de implementación.

¿Qué son Clases en React?

Los componentes de clase en React son la forma antigua en la que se usaban componentes con lógica y estado interno dentro de la biblioteca.

Para poder utilizarlas, el componente debe heredar de la clase base de React de la siguiente forma:

- `class Micomponente extends React.Component.`

Además de que requiere el uso de un método llamado **render()** para imprimir la interfaz en el navegador.

Así se ve el uso de una clase de React para generar un contador:

```

JavaScript

import React from 'react';

// Heredamos de la clase base de React (igual que el "extends" en Java)
class Contador extends React.Component {

  // El constructor para inicializar el "Estado" (los atributos del componente)
  constructor(props) {
    super(props); // Llama al constructor del padre (como el super() de Java)

    this.state = {
      numero: 0 // Este es nuestro atributo/variable de estado
    };
  }

  // Un método de la clase para cambiar el estado
  incrementar = () => {
    this.setState({ numero: this.state.numero + 1 });
  }

  // El método obligatorio que le dice a React qué pintar en pantalla
  render() {
    return (
      <div>
        <p>Clicks: {this.state.numero}</p>
        <button onClick={this.incrementar}>Incrementar</button>
      </div>
    );
  }
}

export default Contador;

```

Principales Eventos en React

Un evento es una acción realizada por el usuario que utilizamos como desencadenante para la ejecución de una respuesta por parte del sistema, como lo pueden ser el hacer click en un botón, hacer scroll en la página, llenar el campo de un formulario etc.

En React los eventos se manejan pasando funciones definidas en JSX. Tienen una regla en cuanto a la sintaxis en donde siempre se deben escribir en formato CamelCase como por ejemplo **onClick**.

Los principales eventos, los cuales se clasifican según su uso son los siguientes:

Eventos del ratón

Son los más comunes ya que capturan la interacción constante del usuario con la página.

- **Evento onClick.** Que se dispara cada vez que el usuario hace click sobre un elemento.

```
<button onClick={() => alert('¡Agregado al carrito!')}>Comprar</button>
```

- **Eventos onMouseEnter y onMouseLeave.** Se activan cada vez que el usuario pasa el curso del mouse sobre el elemento con este evento, similar a **:hover** en CSS.

Eventos de Formularios y Teclado

Se utilizan para lo relacionado con la recolección de datos ingresados por el usuario.

- **Evento onChange.** Se dispara cada vez que el usuario ingresa un carácter en un campo de texto como: **<input>**, **<textarea>** o cuando se selecciona una de las opciones de un campo desplegable.
- **Eventos onKeyDown y onKeyUp.** Se ejecutan cuando el usuario presiona o suelta una tecla.
- **Evento onSubmit.** Este evento esta directamente relacionado con el uso de la etiqueta **<form>** en CSS y se dispara cuando el usuario trata de hacer el envío de un formulario.

Los formularios por defecto recargan toda la página cuando se hace el envío de un formulario; para evitar esto usamos la función de JS **e.preventDefault()** dentro del evento.

